

A Project Report
On
Fuselage — Ground Control Station (GCS)
(Mission Planner UI + Auth Backend)

Platform: Flutter Web (UI) + Node/Express (API) + PostgreSQL (DB)

Generated from the current repository state only (no conceptual features included).

Table of Contents

1. EXECUTIVE SUMMARY
2. BACKGROUND
3. PROJECT OVERVIEW
4. OVERALL PROJECT PLANNING
5. HIGH LEVEL SYSTEM ANALYSIS
6. USE CASE MODEL
7. DESIGN MODEL
8. TESTING
9. TRANSITION
10. ANNEXURE
11. CONCLUSION

1. EXECUTIVE SUMMARY

Fuselage is a Flutter-based Ground Control Station (GCS) user interface implemented as a map-first mission planner shell. It includes a top status bar, left navigation sidebar, right telemetry panel, and bottom mission dock. Telemetry and mission state are simulated/UI-only in the current build; a real flight controller link (MAVLink), calibration execution, and mission upload/download are not implemented.

2. BACKGROUND

2.1 Existing System

Traditional mission planning and field monitoring workflows often rely on disconnected tools (maps, checklists, and manual status reporting). This can lead to fragmented situational awareness and inconsistent operator workflows.

2.2 Definition of Problem

This milestone focuses on providing a stable, operator-friendly GCS UI shell that standardizes navigation, map overlays, mission controls, and telemetry presentation in one interface.

2.3 System (Implemented Scope)

Fuselage delivers a polished UI shell with simulation-driven telemetry and mission state. The backend currently provides authentication endpoints and a single persisted table (users) in PostgreSQL.

3. PROJECT OVERVIEW

3.1 Objective

To implement a map-first GCS mission planner interface (UI) with clear operational layout and role-ready authentication, suitable for extension to real vehicle connectivity later.

3.2 Stakeholders

- Pilot/Operator
- Admin
- Supervisor/Operations Officer
- Field Technician

3.3 Scope (Current Build)

Includes: map rendering, waypoint UX (demo), map layers toggles, telemetry panels (simulated), alerts UI, mission dock controls (some placeholders), and a tools sheet with navigable routes. Excludes: MAVLink transport, mission upload/download to autopilot, calibration workflows, and real command/ack handling.

4. OVERALL PROJECT PLANNING

4.1 Development Environment

Frontend: Flutter (Dart) with flutter_map, geolocator, camera, shared_preferences, http. Backend: Node.js (Express) with pg (PostgreSQL), bcryptjs, jsonwebtoken, dotenv, cors.

4.2 Constraints

The current release is UI-first: telemetry, link quality, and mission actions are simulated or placeholders. Real flight controller integration and safety-critical command pipelines are not present.

4.3 Deliverables

- Flutter GCS UI shell (map-first)
- Node/Express auth backend
- PostgreSQL schema for users
- Deployment configuration for Railway (API/DB) and Vercel (web)

4.4 Risks

- Transition from simulated telemetry to live MAVLink may introduce timing/protocol edge cases.
- High-frequency telemetry and overlays can stress lower-end devices; web performance requires throttling.

5. HIGH LEVEL SYSTEM ANALYSIS

5.1 User Characteristics

Users are expected to have basic familiarity with web/app navigation and map interactions, including login, using menus, reading status indicators, and following safety prompts.

5.2 Summary of System Features / Functional Requirements

- User authentication (register/login) via API
- Map-first GCS shell (top bar, sidebar, telemetry panel, bottom dock)
- Map actions: add waypoint (demo), layers toggles, flight mode selection (demo), mission wizard route
- Telemetry deck and alerts UI (simulated)
- Tools sheet routes (telemetry, camera, manual control, airspace, replay, plugins, vehicles, setup)

5.3 Non-Functional Requirements (Observed in Build)

- Usability: consistent tactical UI and clear grouping around the map workspace
- Responsiveness: clamp widths and avoid overflows; sidebar collapsed by default
- Security: hashed passwords and JWT-based auth on backend
- Maintainability: modular widget structure and route-based navigation

5.4 Glossary

Term	Description
GCS	Ground Control Station software for monitoring/control.
Mission	A planned route defined by waypoints.
Waypoint	A lat/long point in a mission route.
Telemetry	Live flight metrics (simulated in current build).
NFZ	No-fly zone overlay used for demo start gating.
RTH	Return-to-home mode (UI feedback only).

6. USE CASE MODEL

6.1 Use Case Text (Implemented)

Login/Register: user authenticates via API and enters the GCS shell. Map operations: user views map, toggles layers, adds waypoints (demo), and monitors telemetry/alerts (simulated).

6.2 Operation Contracts (Auth API)

Operation	Preconditions	Postconditions
POST /auth/register	name/email/password provided; email unique	user inserted into users; token returned
POST /auth/login	user exists; password correct	JWT token returned; user payload returned
GET /health	database reachable	returns status ok and db up

7. DESIGN MODEL

7.1 UI Design (Current Screens)

- Splash: loading sequence into Auth
- Auth: Login + Register
- GCS Shell: Top bar, Sidebar (Map/Mission/Logs/Settings/Tools), Map workspace, Right telemetry panel, Bottom dock
- Tools routes: Telemetry, Camera, Manual Control, Airspace, Replay, Plugins, Vehicles, Vehicle setup (some scaffold-only)

7.2 Entity Relationship Diagram (Actual DB)

The implemented database currently contains only the users table.

Field	Type	Constraints
id	BIGSERIAL	PK
name	TEXT	NOT NULL
email	TEXT	NOT NULL, UNIQUE
password_hash	TEXT	NOT NULL
pilot_id	TEXT	NULL
role	TEXT	NOT NULL, DEFAULT 'pilot'
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

7.3 Theoretical Background (Actual Technologies Used)

The frontend is built in Flutter (Dart) using a widget-driven UI architecture and route-based navigation. The map workspace uses flutter_map with overlay widgets for mission controls and telemetry panels. The backend is a Node.js/Express API that implements JSON request/response endpoints for authentication and health checks, using JWT for session tokens and bcrypt for password hashing. PostgreSQL stores user records with constraints to ensure uniqueness and integrity.

8. TESTING

Testing in the current repository includes manual verification of UI flows and API endpoints:

- API health check verifies DB connectivity (GET /health).
- Registration and login verify password hashing and JWT token issuance.
- UI interactions verify map rendering, telemetry panel toggles, and bottom dock behavior (simulation-driven).

9. TRANSITION

9.1 System Implementation (Deployed Setup)

Backend and database are deployed on Railway (API service + Postgres). The Flutter web frontend is deployed on Vercel. The frontend is built with `AUTH_API_URL` pointing to the Railway API origin.

10. ANNEXURE

Repository documents used for this report:

- docs/GCS_Report.md
- docs/GCS_User_Manual.md
- backend/sql/init.sql
- backend/src/server.js and backend/src/auth.js

11. CONCLUSION

Fuselage provides a functional, polished GCS UI shell with map rendering, structured overlays, and simulation-driven telemetry. The backend supports secure user authentication and persists users in PostgreSQL. Real flight-controller integration (MAVLink) and mission protocol operations are not implemented in this milestone.